호텔 예약 시스템 설계

가상 면접 사례로 배우는 대규모 시스템 설계 기초 2

오늘의 질문

남은 방은 1개,
두 사람이 동시에
예약 버튼을 눌렀다

둘 다 성공하면 방이 없는 손님이 생기고,
둘 다 실패하면 빈 방을 못 판다

목차

01 요구사항

02 규모 추정

03 API 설계

04 데이터 모델

05 개략적 설계

06 상세 설계

07 동시성

08 규모 확장

09 데이터 일관성

정리와 Q&A

01 요구사항

고객이 움직이는 순서대로 필요한 것을 뽑아본다면

- 고객 흐름 따라가기
- 기능 요구사항
- 비기능 요구사항

고객이 움직이는 순서대로 요구사항 도출



단계마다 무엇을 보여줘야 하는지와 어떤 품질로 동작해야 하는지를 나눠서 정리

기능 요구사항



규모

호텔 5,000개, 객실 100만 개



결제

예약할 때 전액 결제



채널

웹사이트에서만 예약



취소

예약 취소 가능



초과 예약

객실 수의 110%까지 예약



요금

날짜마다 가격이 다르다

비기능 요구사항

높은 동시성

성수기나 큰 행사가 있으면
인기 호텔 특정 객실에 예약이 몰린다

적당한 응답 시간

빠르면 좋지만
예약 처리에 몇 초는 괜찮다

속도를 조금 양보하는 대신 정확성을 챙길 여유가 있다

02 규모 추정

숫자를 먼저 뽑아 두면 뒤의 선택이 쉬워진다

- 초당 예약 수 (TPS)
- 페이지별 조회 수 (QPS)
- 평균과 피크, 그리고 핫스팟

하루 24만 건, 초당 3건

24만

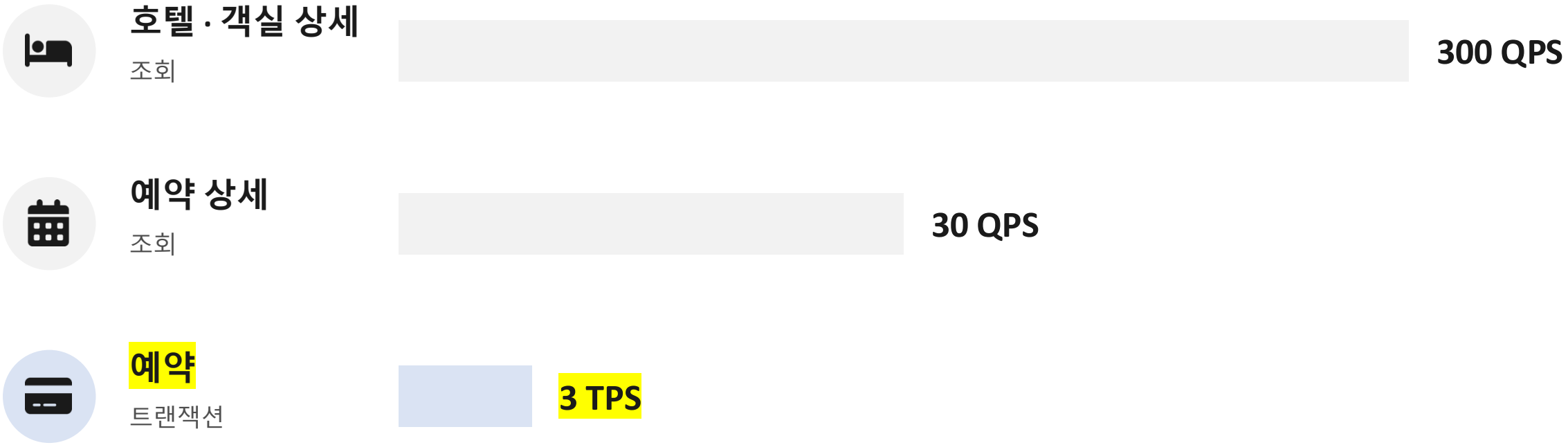
하루 예약 건수
100만 × 0.7 ÷ 3

3 TPS

초당 예약
24만 ÷ 86,400초

가정: 객실 점유율 70%, 평균 3박

예약 한 번에 조회는 100번



단계마다 10%만 다음으로 넘어간다고 보고 예약 TPS 3에서 거꾸로 계산 (막대 길이는 개념 표시)

읽기와 쓰기의 비율

읽기 100 : 쓰기 1



DB, 락, 캐시를 고를 때마다
이 숫자로 돌아온다

평균과 피크

평균 3 TPS는
크리스마스 이브에도
3일까

성수기 피크는 평소의 몇 배쯤 될까요?



피크를 대충 계산해 봤다



행사 당일

하루 예약의 30%가 오픈 직후 1시간에 몰리면
약 20 TPS, 평균의 7배



콘서트장 근처 호텔

호텔 10곳에 1분 동안 1만 명
초당 170건이 몇 개 행에 몰린다



전체 양보다 한곳에 몰리는 정도가 더 문제다

핫스팟

한 행에 요청이 몰리면
그 행이 병목이 된다



이런 곳을 핫스팟이라고 부른다.
07장 동시성에서 다시 나온다.

03 API 설계



호텔, 객실, 예약 세 가지 리소스

- REST API 목록
- 예약 생성 요청과 멱등 키

API는 평범한 REST로 잡았다

리소스	조회	생성	갱신 · 삭제
호텔	GET /v1/hotels/:id	POST /v1/hotels	PUT · DELETE /v1/hotels/:id
객실	GET /v1/hotels/:id/rooms/:id	POST /v1/hotels/:id/rooms	PUT · DELETE …/rooms/:id
예약	GET /v1/reservations/:id	**POST /v1/reservations**	PUT · DELETE /v1/reservations/:id

✔ **눈여겨볼 건 예약 생성 하나다**

책에 정리된 API 목록

호텔

API	Detail
GET /v1/hotels/ID	Get detailed information about a hotel.
POST /v1/hotels	Add a new hotel. This API is only available to hotel staff.
PUT /v1/hotels/ID	Update hotel information. This API is only available to hotel staff.
DELETE /v1/hotels/ID	Delete a hotel. This API is only available to hotel staff.

객실

API	Detail
GET /v1/hotels/ID/rooms/ID	Get detailed information about a room.
POST /v1/hotels/ID/rooms	Add a room. This API is only available to hotel staff.
PUT /v1/hotels/ID/rooms/ID	Update room information. This API is only available to hotel staff.
DELETE /v1/hotels/ID/rooms/ID	Delete a room. This API is only available to hotel staff.

예약

API	Detail
GET /v1/reservations	Get the reservation history of the logged-in user.
GET /v1/reservations/ID	Get detailed information about a reservation.
POST /v1/reservations	Make a new reservation.
DELETE /v1/reservations/ID	Cancel a reservation.

예약 생성 요청에 ID가 들어 있다

```
POST /v1/reservations
{
  "reservationId": "13422445",
  "hotelId": "245",
  "roomTypeId": "U12354673389",
  "startDate": "2026-04-28",
  "endDate": "2026-04-30"
}
```



생성 요청인데
왜 ID를 클라이언트가 보낼까?

이 ID가 멱등 키다.

04 데이터 모델



어떻게 읽고 쓰는지 보고 DB를 고르다

- 데이터 접근 패턴
- 관계형 DB를 고른 이유
- 예약 상태

데이터를 어떻게 읽고 쓰는지부터 봤다

호텔 상세 조회

호텔과 객실 정보를 보여준다

예약 가능한 객실 조회

지정한 날짜에 남은 방이 있는지 확인한다

예약 저장 · 조회 · 취소

예약 기록을 만들고 관리한다

관계형 DB를 골랐다



읽기가 훨씬 많다

읽기 위주 흐름에 잘 맞는다. NoSQL은 쓰기 쪽에 강한 경우가 많다



ACID 트랜잭션

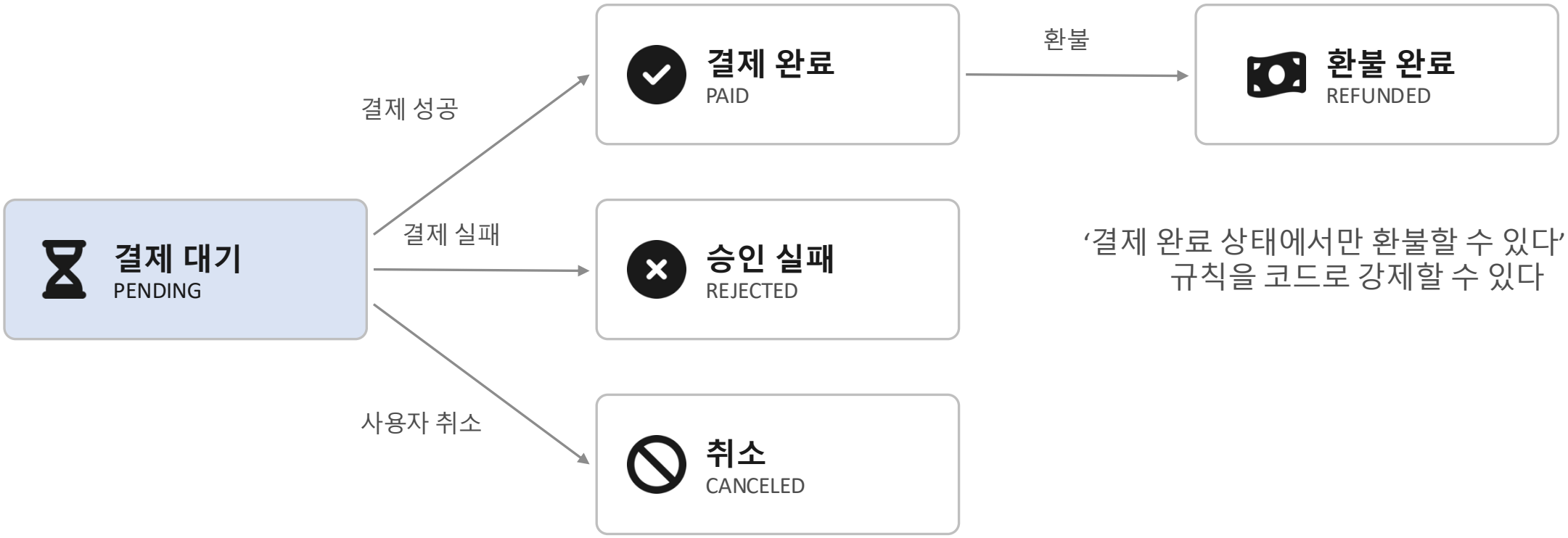
재고 차감과 예약 생성이 같이 성공하거나 같이 실패해야 한다



구조가 명확하다

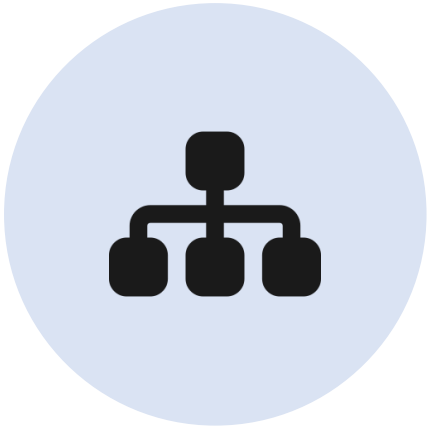
호텔, 객실, 예약, 투숙객의 관계가 잘 바뀌지 않는다

예약에는 상태가 있다



‘결제 완료 상태에서만 환불할 수 있다’ 같은
규칙을 코드로 강제할 수 있다

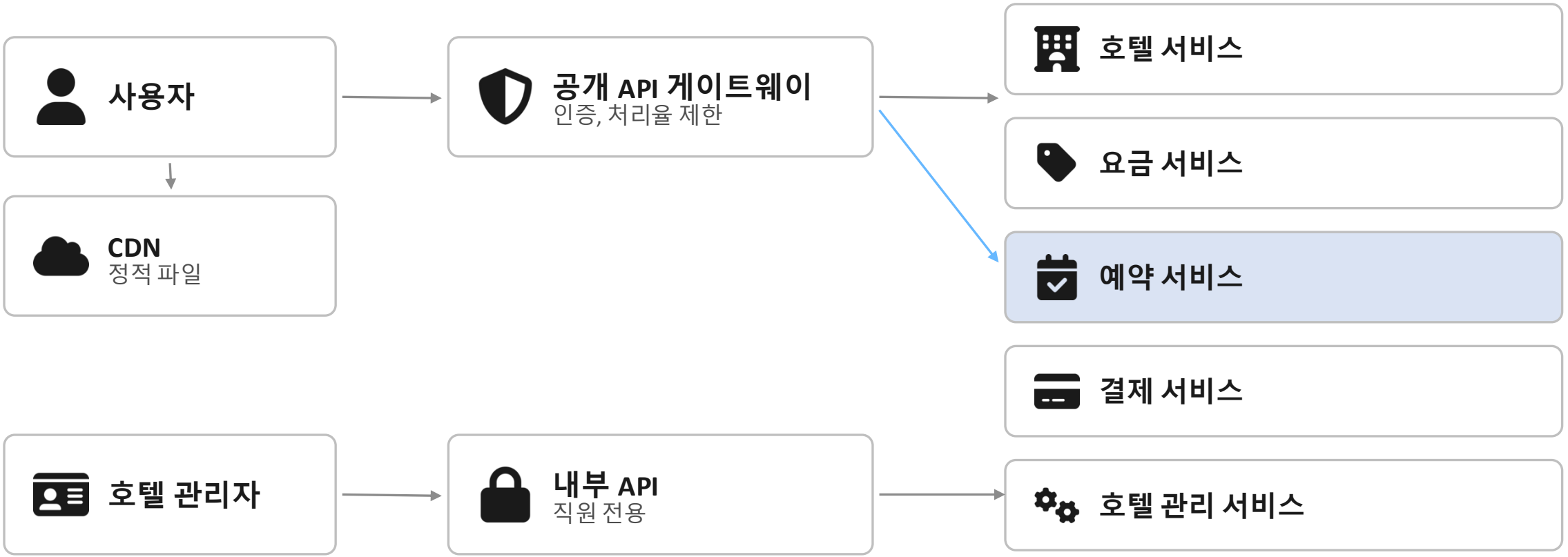
05 개략적 설계



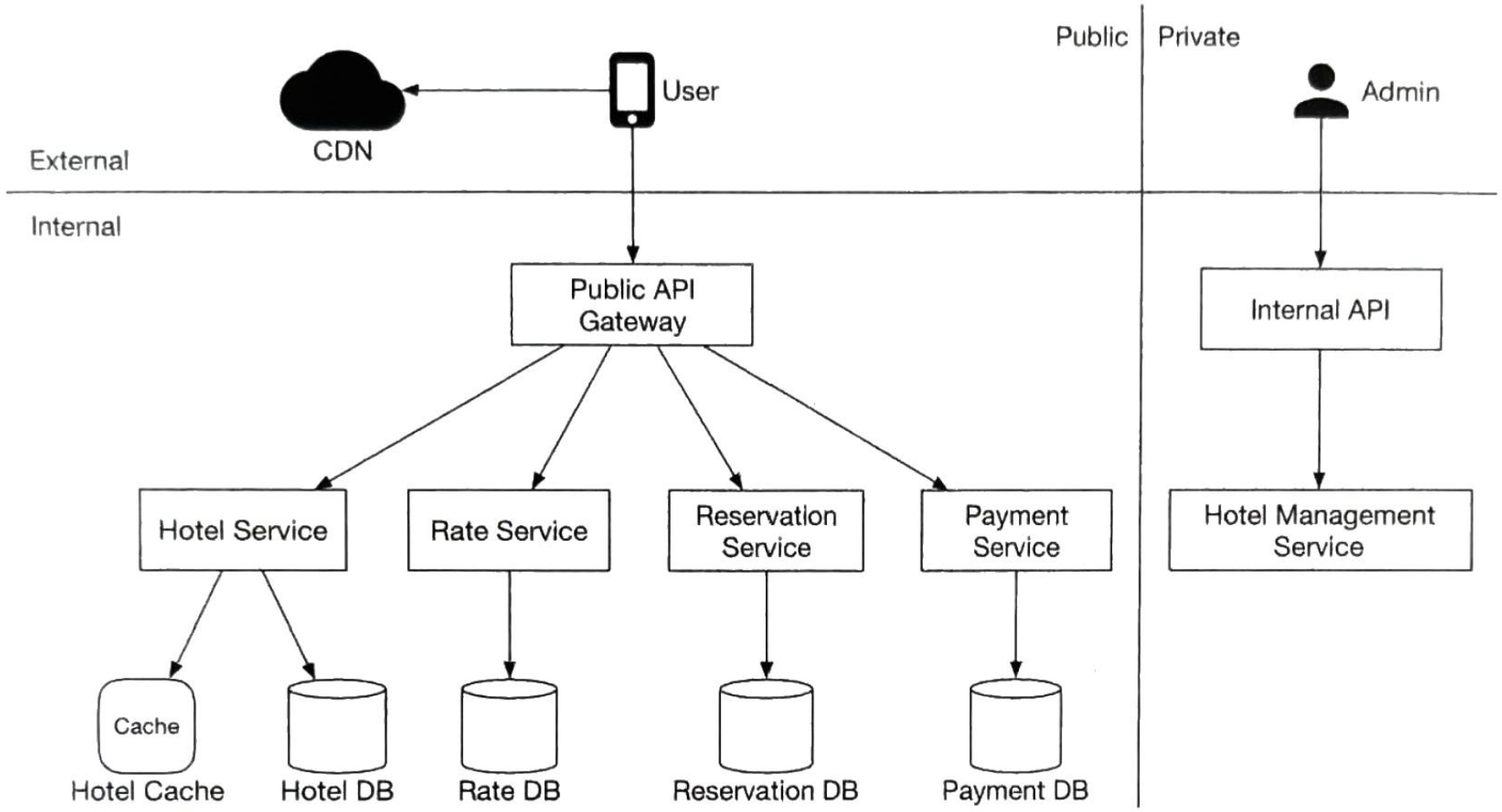
마이크로서비스와 서비스 간 통신

- 전체 구조
- gRPC를 고른 이유
- 동기와 비동기

서비스를 역할별로 나눴다



책에 나온 개략적 설계도


Public
Private
User
Admin
CDN
External
Internal
Public API Gateway
Internal API
Hotel Service
Rate Service
Reservation Service
Payment Service
Hotel Management Service
Cache
Hotel Cache
Hotel DB
Rate DB
Reservation DB
Payment DB

서비스끼리는 gRPC로 통신한다



이진 직렬화

Protobuf를 써서 JSON보다 작고 빠르다



HTTP/2

연결 하나로 여러 요청을 보내고, 헤더도 압축한다



타입 체크

.proto 파일로 스키마를 정하고 코드를 생성한다

다른 방식과 비교해 보면

	gRPC	REST	GraphQL	메시지 큐
직렬화	Binary (Protobuf)	Text (JSON)	Text (JSON)	Binary / Text
프로토콜	HTTP/2	HTTP/1.1	HTTP/1.1, 2	자체 프로토콜
통신 방식	동기 / 스트림	동기 요청-응답	동기	비동기 Pub/Sub
타입 안정성	매우 높음	낮음	높음	보통
성능	매우 높음	보통	보통	높음 (처리량)

동기와 비동기를 섞어 썼다



바로 답이 필요할 때

gRPC 동기 호출
예약할 때 요금 조회, 결제 요청



알리기만 하면 될 때

Kafka 이벤트
예약이 생긴 뒤 투숙객 정보, 캐시 갱신



실제 서비스도 대부분 이렇게 나눠서 쓴다

06 상세 설계



객실 유형 단위 예약과 재고 테이블

- 예약 단위 바꾸기
- 재고 테이블과 초과 예약
- DDL, 인덱스, 요금
- 결제 대기 예약의 만료

예약 단위

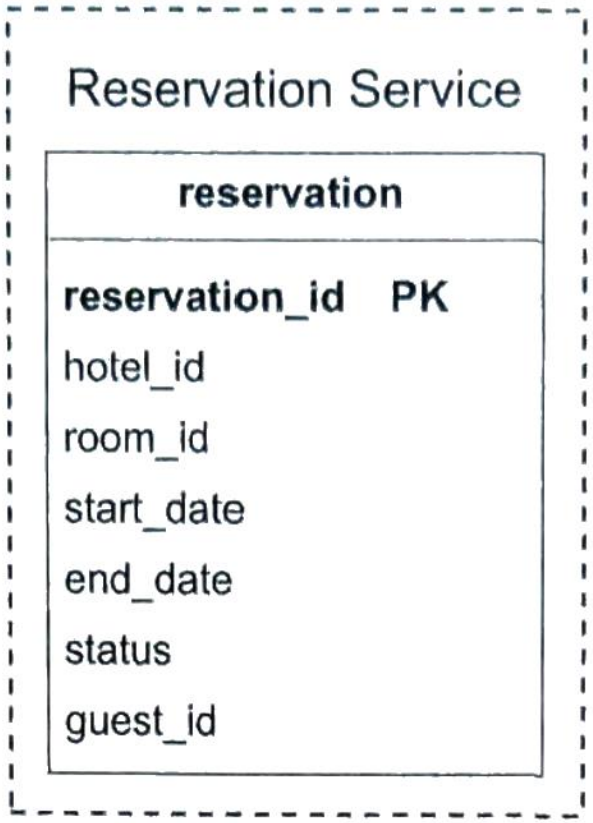
호텔을 예약할 때
101호를 고르지는 않는다



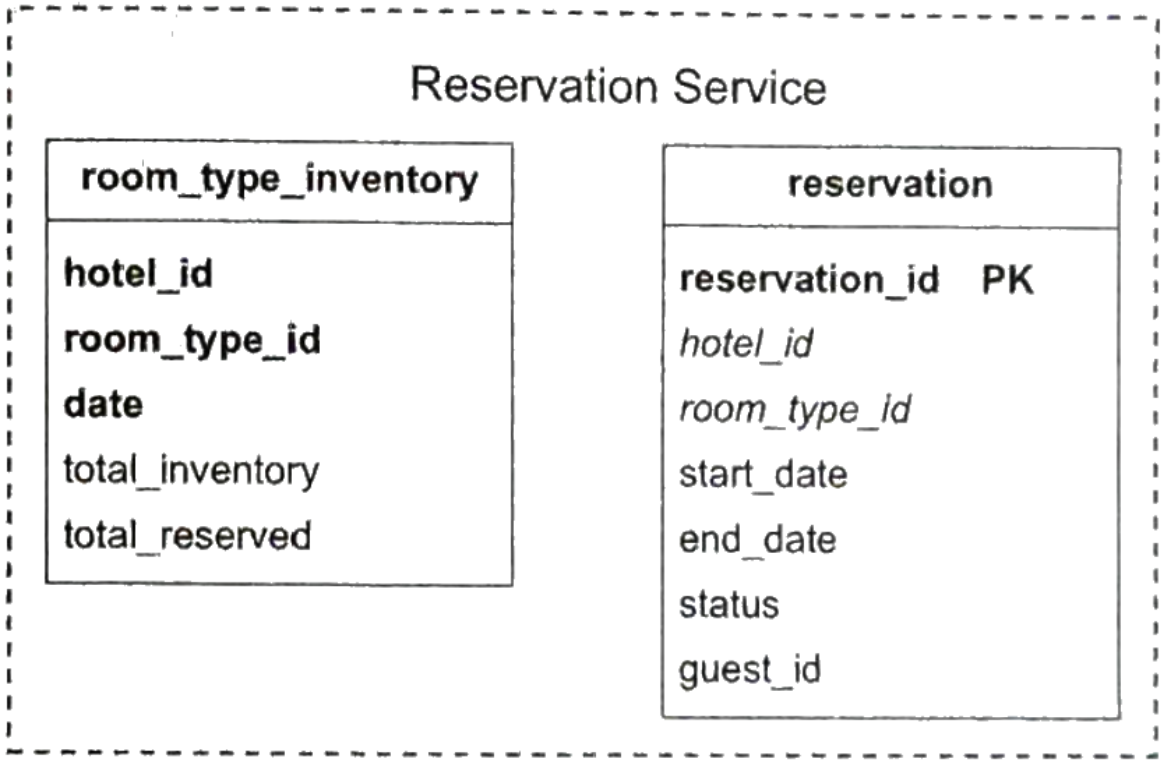
고르는 건 ‘디럭스 더블’ 같은 객실 유형이다.
그래서 예약 단위를 객실 유형으로 바꿨다.

room_id 대신 room_type_id로 바꿨다

바꾸기 전



바꾼 후



재고는 객실 유형과 날짜마다 한 줄

hotel_id	room_type_id	date	total_inventory	total_reserved
211	1001	2026-01-01	100	97
211	1001	2026-01-02	100	87
211	1001	2026-01-03	100	77



기본키 = (hotel_id, room_type_id, date)

남은 방 확인은 한 줄 조회, 3박이면 세 줄을 본다

10% 초과 예약은 조건 하나로 끝난다

(total_reserved + 예약할 객실 수) <= 1.1 × total_inventory

97 + 3 = 100 ≤ 110

1월 1일에 3개를 예약하려는 경우. 초과 예약이 없었다면 거의 한계였을 상황이다.

2년치 재고를 미리 만들면 7,300만 행

7,300만

5,000 호텔 × 20 객실 유형 × 730일

- DB 한 대에 들어가는 크기
- 한 대가 죽으면 끝이라 리전별로 복제
- 더 커지면 지난 날짜는 아카이빙
- hotel_id 기준으로 샤딩 (08장)

실제 테이블은 이렇게 생겼다

```
CREATE TABLE room_type_inventory (
  hotel_id       BIGINT NOT NULL,
  room_type_id   BIGINT NOT NULL,
  date           DATE   NOT NULL,
  total_inventory INT   NOT NULL,
  total_reserved  INT   NOT NULL,
  version        BIGINT NOT NULL,  -- 낙관적 락
  PRIMARY KEY (hotel_id, room_type_id, date)
);
```

```
CREATE TABLE reservation (
  reservation_id VARCHAR(64) PRIMARY KEY,
  hotel_id       BIGINT NOT NULL,
  room_type_id   BIGINT NOT NULL,
  guest_id       BIGINT NOT NULL,
  start_date     DATE NOT NULL,
  end_date       DATE NOT NULL,
  status         VARCHAR(20) NOT NULL,
  INDEX idx_guest (guest_id),
  INDEX idx_hotel_date (hotel_id, start_date)
);
```

인덱스는 조회 패턴을 따라간다

키 / 인덱스	어떤 조회를 위한 것인가	효과
재고 PK (hotel, type, date)	211번 호텔 1001 유형, 1/1 ~ 1/3	PK 범위 스캔 한 번
reservation PK	멱등 키 중복 확인	유일성 자동 보장
idx_guest	내 예약 목록	guest_id로 조회
idx_hotel_date	이번 주 체크인 목록 (관리자)	hotel_id + start_date

복합 키는 왼쪽부터 쓰이니, 가장 자주 고정되는 호텔을 앞에 뒀다

요금도 날짜마다 한 줄

```
CREATE TABLE room_type_rate (
  hotel_id     BIGINT NOT NULL,
  room_type_id BIGINT NOT NULL,
  date         DATE   NOT NULL,
  rate         DECIMAL(10,2),
  PRIMARY KEY (hotel_id, room_type_id, date)
);
-- 3박 요금 = 세 줄의 합
```

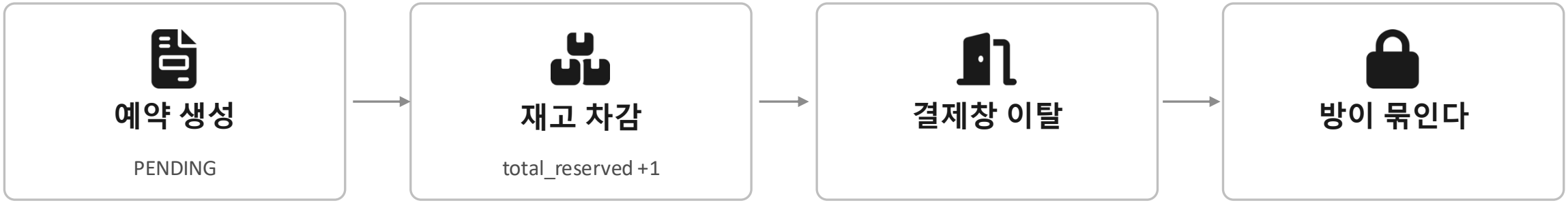


결제 대기 중에 가격이 바뀌면?

예약할 때 금액을 예약 레코드에 같이 저장한다

요금 테이블은 지금 가격, 예약은 고객이 동의한 가격

결제하지 않고 나간 사람의 방



결제 대기 예약에 만료 시간을 둔다

예를 들어 15분 안에 결제하지 않으면 만료

만료되면 취소하고 재고를 돌려놓는다

스케줄러나 지연 메시지로 처리한다. 이때도 동시성 제어가 필요하다

07 동시성



이중 예약을 어떻게 막을까

- 멱등 키
- 경쟁 조건
- 비관적 락과 낙관적 락
- 격리 수준, 조건부 UPDATE, 핫스팟

이중 예약은 두 경우에 생긴다



같은 사람이 여러 번

버튼을 연타하거나
타임아웃으로 자동 재시도



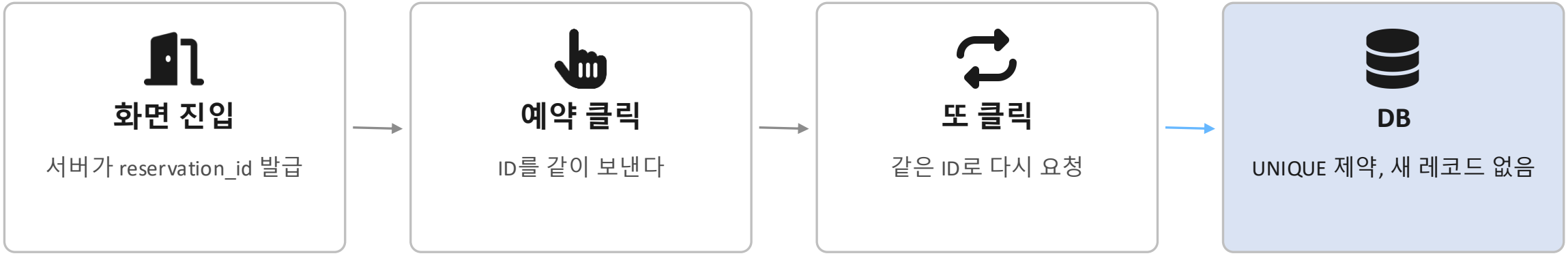
다른 사람이 동시에

마지막 남은 방을
두 명이 동시에 예약



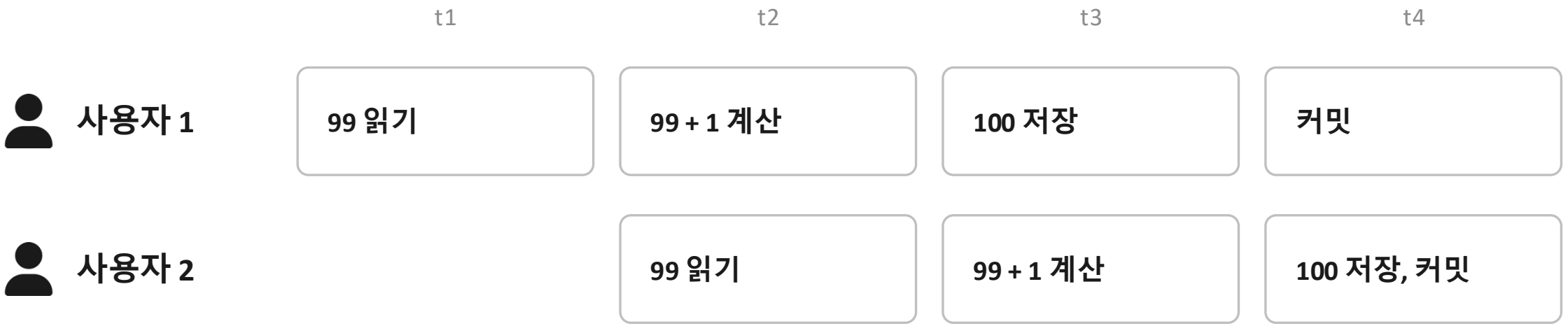
앞은 멱등 키로, 뒤는 락으로 막는다

같은 요청이 여러 번 와도 예약은 한 번



서버가 요청마다 새 ID를 만들면 중복 요청을 구분할 수 없다.

둘 다 99를 읽었다



예약은 2건인데 재고는 1만 늘었다

SERIALIZABLE로 올리면 막히지만 전체가 느려진다. 그래서 필요한 곳에만 락을 쓴다.

비관적 락: 읽을 때부터 잠근다

```
BEGIN;
SELECT total_reserved, total_inventory
  FROM room_type_inventory
 WHERE hotel_id = 211 AND room_type_id = 1001 AND date = '2026-01-01'
  FOR UPDATE;          -- 사용자 2는 여기서 기다린다
UPDATE room_type_inventory SET total_reserved = total_reserved + 1 WHERE ...;
INSERT INTO reservation ...;
COMMIT;                -- 락이 풀리면 사용자 2는 100을 읽고 실패
```

그림으로 보는 비관적 락

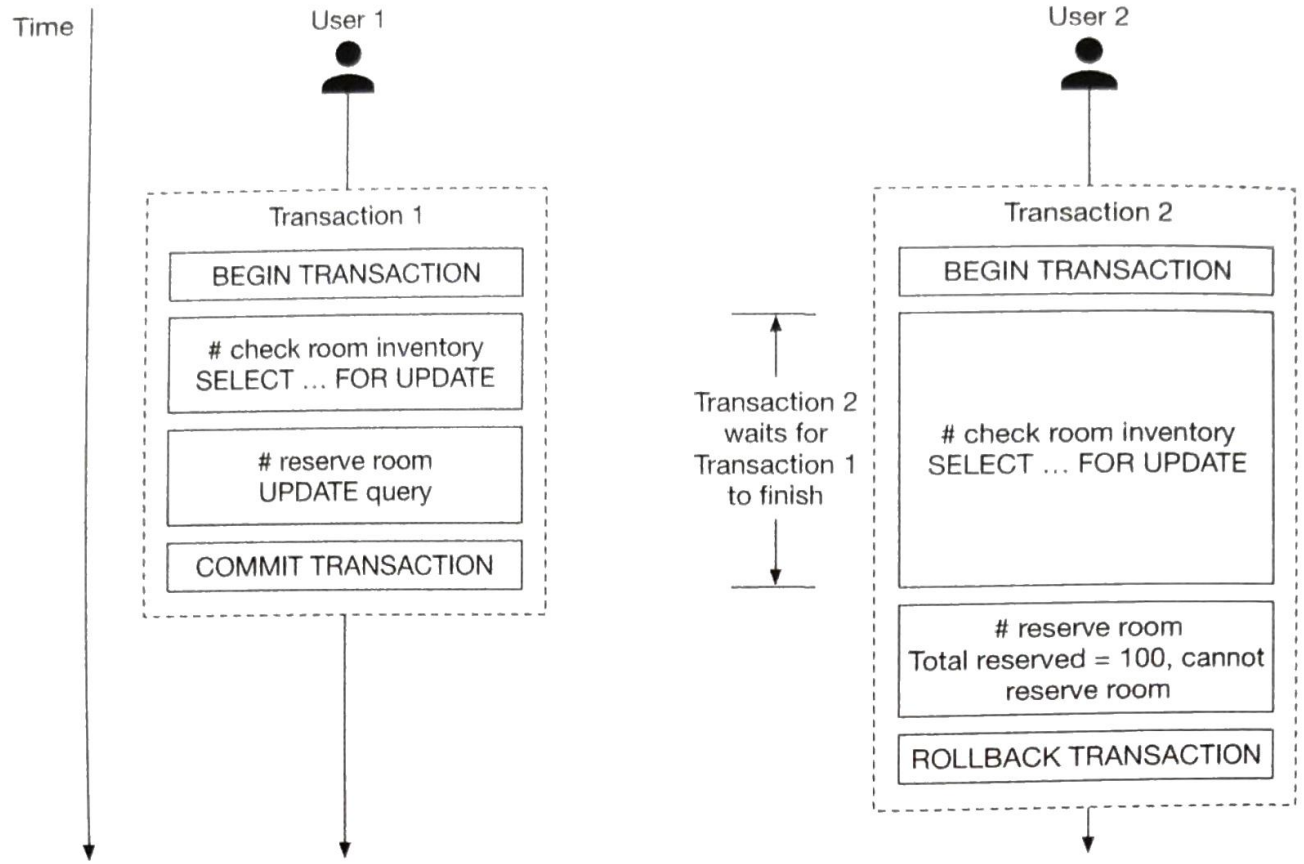


Figure 7.12: Pessimistic locking

비관적 락의 장단점



좋은 점

구현이 쉽다
충돌을 확실히 막는다
경합이 심할 때 쓸 만하다



아쉬운 점

여러 행을 잠그면 데드락이 생길 수 있다
락을 오래 잡으면 확장이 어렵다

낙관적 락: 버전을 보고 갱신한다

```
-- 1) 락 없이 읽는다 → total_reserved = 99, version = 7
SELECT total_reserved, version FROM room_type_inventory WHERE ...;

-- 2) 버전이 그대로일 때만 갱신한다
UPDATE room_type_inventory
   SET total_reserved = 100, version = 8
 WHERE hotel_id = 211 AND room_type_id = 1001 AND date = '2026-01-01'
   AND version = 7;

-- 바뀐 행이 0개면 누가 먼저 바꾼 것 → 다시 읽고 재시도
```

그림으로 보는 낙관적 락

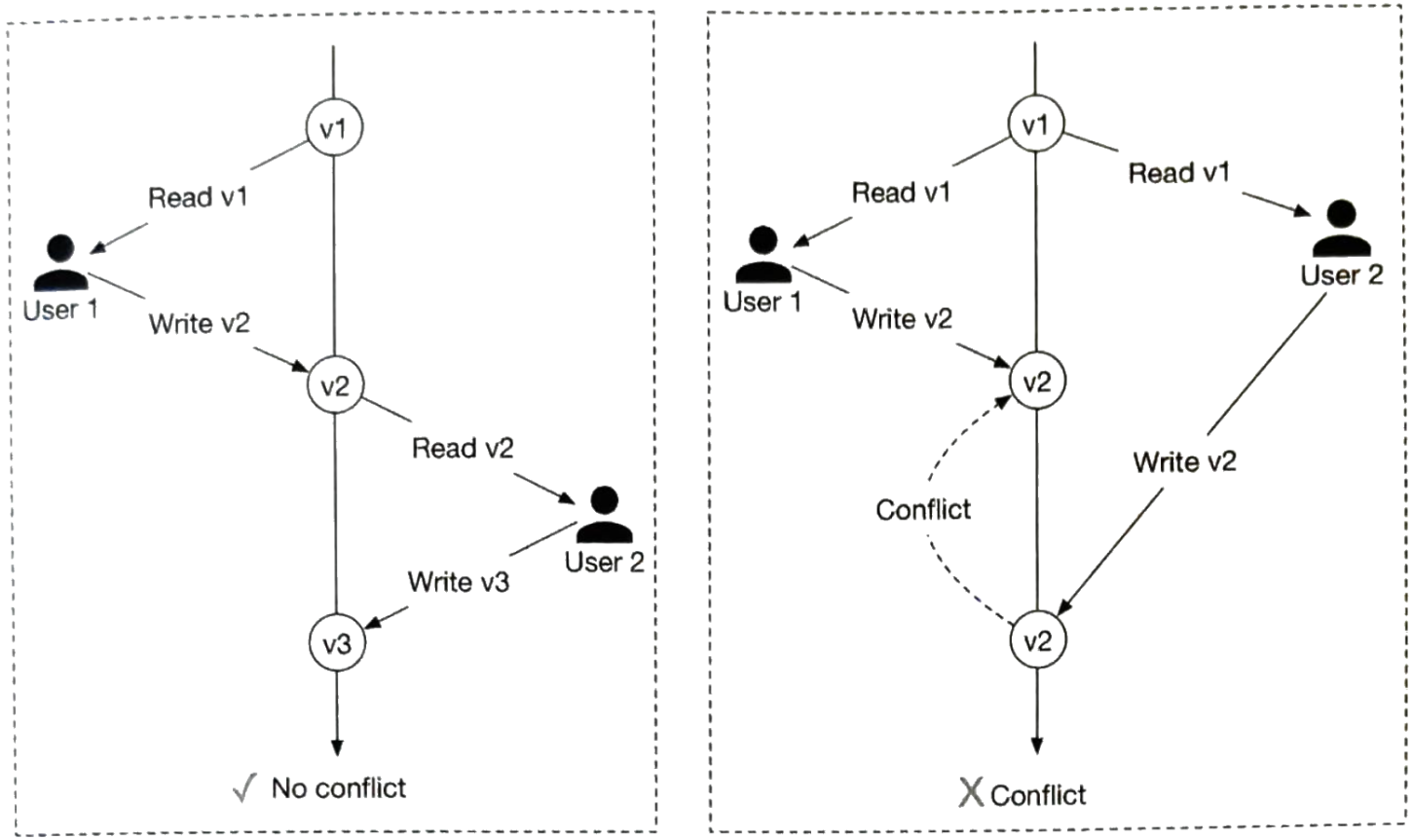


Figure 7.13: Optimistic locking

낙관적 락의 장단점



좋은 점

DB 락을 쓰지 않아서
보통 더 빠르다



아쉬운 점

경합이 심하면 급격히 느려진다
100명이 동시에 누르면
1명만 성공, 99명은 재시도

두 방식을 나란히 놓으면

	비관적 락	낙관적 락 (우리의 선택)
방식	읽을 때 잠그고, 다른 트랜잭션은 대기	잠그지 않고, 버전으로 충돌 확인
충돌하면	기다린다	실패하고 재시도
잘 맞는 상황	경합이 심할 때	경합이 낮을 때
주의할 점	데드락, 확장성	경합 시 재시도 폭증

우리의 선택

초당 3건이면
부딪칠 일이 거의 없다



그래서 낙관적 락을 골랐다

3박이면 세 줄을 한 번에



한 트랜잭션으로 처리한다

세 날짜를 모두 갱신하고, 하나라도 실패하면 전부 롤백한다



잠그는 순서를 맞춘다

비관적 락이라면 항상 날짜 오름차순으로 잠가서 데드락을 피한다

DB 제약 조건으로 막는 방법도 있다

```
CONSTRAINT check_room_count
CHECK ((total_inventory - total_reserved) >= 0)
```



좋은 점

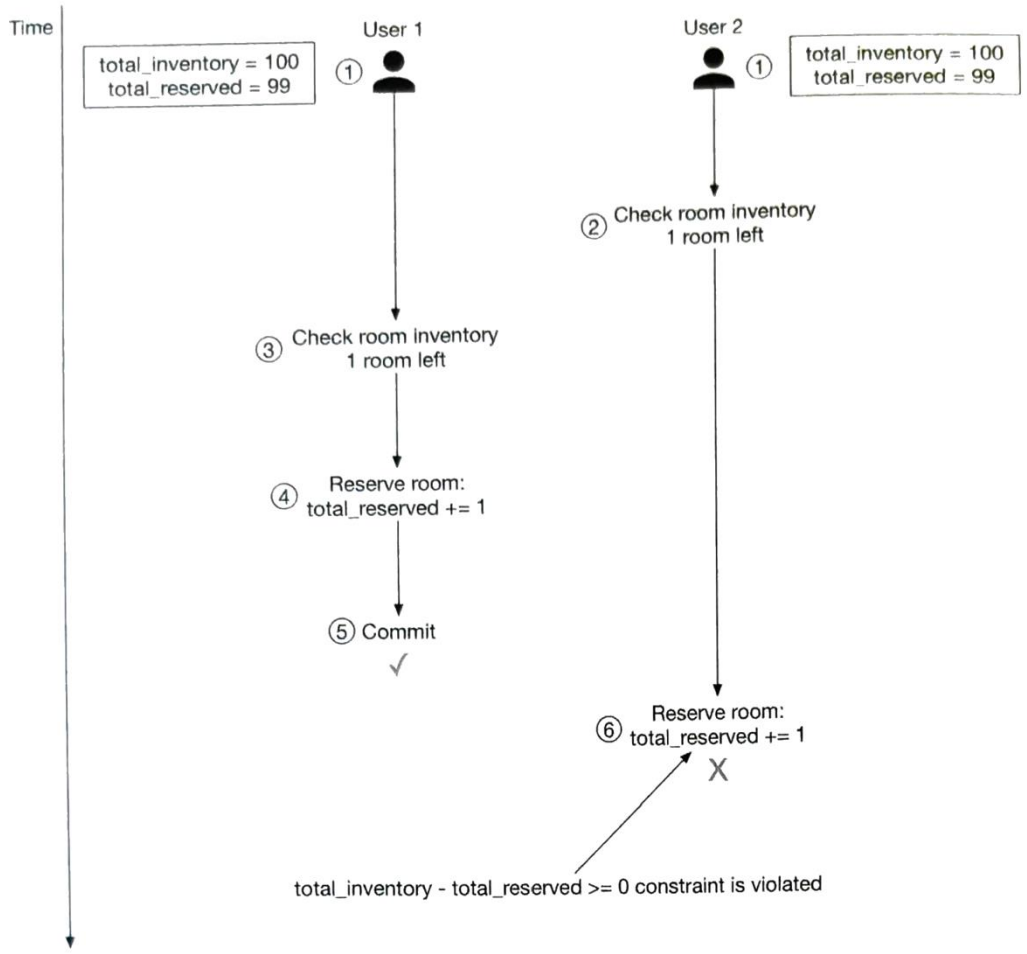
구현이 가장 쉽다
낙관적 락처럼 동작한다



아쉬운 점

경합이 심하면 실패가 많다
모든 DB가 지원하지는 않는다

그림으로 보는 DB 제약 조건



REPEATABLE READ도 갱신 손실은 못 막는다

격리 수준	막아 주는 것	그래도 남는 문제
READ UNCOMMITTED	없음	커밋 안 된 값을 읽는다
READ COMMITTED	Dirty Read	두 번 읽으면 값이 다를 수 있다
REPEATABLE READ (MySQL 기본)	+ Non-repeatable Read	**읽고, 계산하고, 덮어쓰면 갱신이 사라진다**
SERIALIZABLE	거의 전부	느리다

PostgreSQL은 같은 상황에서 에러를 낸다. DB마다 실제 동작이 다르다.

읽지 않고, 조건을 걸어 바로 갱신한다

```
UPDATE room_type_inventory
   SET total_reserved = total_reserved + :rooms
 WHERE hotel_id = :hotelId AND room_type_id = :typeId
   AND date BETWEEN :start AND :lastNight
   AND total_reserved + :rooms <= total_inventory * 1.1;

-- 바뀐 행 수 == 숙박일 수 → 성공
-- 더 적으면 어느 날짜가 매진 → 롤백
```

✔ **확인과 차감이 한 문장에 끝난다**

한 행에 요청이 몰릴 때

방법	어떻게	대신 감수할 것
요청 대기열 (큐)	같은 호텔·유형 요청을 줄 세워 하나씩	응답이 비동기가 된다
Redis 원자 연산	DECRBY, Lua로 먼저 차감하고 통과한 것만 DB로	Redis와 DB 재고를 맞춰야 한다
분산 락	hotel:type:date 키로 잠금	락 만료, 장애 처리를 설계해야 한다
대기실	들어오는 사람 수를 제한	사용자가 기다린다

상황마다 다르게 고른다



평소

낙관적 락이나 조건부 UPDATE로 충분하다



행사가 예정됐을 때

Redis에서 먼저 걸러내고 DB에서 최종 확인한다



콘서트급 오픈

대기실과 큐를 둔다



어느 쪽이든 마지막 판단은 DB가 한다

08 규모 확장



트래픽이 1,000배가 되면 고려해야 할 것들

- DB 샤딩
- 캐시와 CDC
- 캐시 전략, 스탬피드, 리샤딩

트래픽이 1,000배가 되면



애플리케이션 서버

상태가 없어서
서버를 늘리고 로드밸런서를 붙이면 된다



데이터베이스

모든 서버가 같은 DB를 본다
여기가 먼저 막힌다

hotel_id로 DB를 나눈다



조회할 때 가장 많이 쓰는 값으로 나눈다

거의 모든 조회가 '이 호텔의 이 날짜'라서 hotel_id

예약 트랜잭션이 샤드 하나에서 끝난다

30,000 QPS ÷ 16 ≈ 샤드당 1,875 QPS

캐시는 TTL과 LRU로 관리한다



TTL

지난 날짜는 거의 안 본다
오래된 데이터는 자동으로 지운다

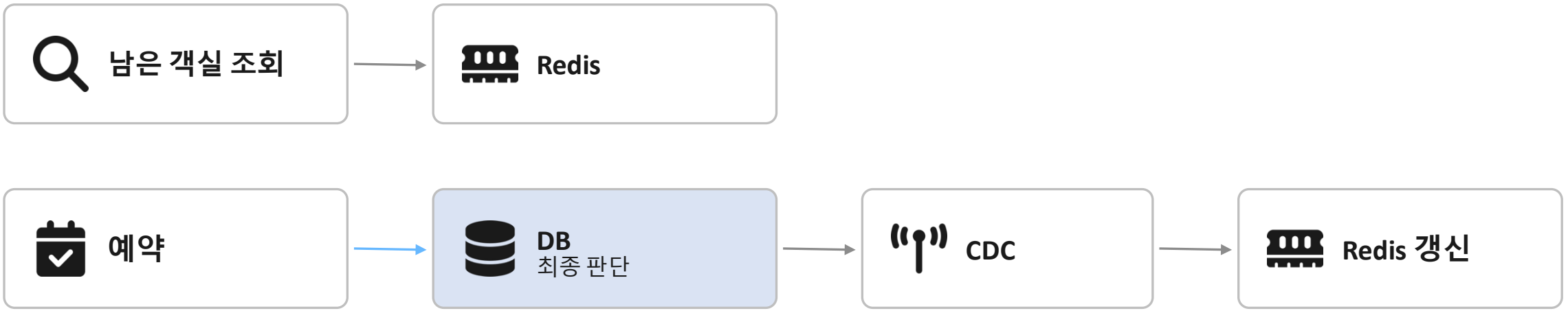


LRU

메모리가 차면
가장 오래 안 쓴 것부터 지운다

LRU는 접근할 때마다 순서를 옮기는 비용이 든다. 다른 정책으로 LFU, FIFO도 있다.

읽기는 캐시, 쓰기는 DB



캐시가 잠깐 틀려도 이중 예약은 생기지 않는다

예약할 때 DB에서 다시 확인하니까. 최악이면 '있는 줄 알았는데 예약 실패' 정도다.

DB의 변경을 CDC로 전파한다



애플리케이션이 DB와 캐시에 따로 쓰는 것보다 빠뜨릴 일이 적다

캐시를 채우는 방법들

전략	동작	좋은 점	아쉬운 점
Cache-Aside	없으면 DB에서 읽어 채움	단순하다	첫 요청이 느리다
Write-Through	쓸 때 DB와 캐시에 같이	캐시가 최신에 가깝다	쓰기가 느려진다
Write-Behind	캐시에 먼저, DB는 나중에	쓰기가 빠르다	캐시가 죽으면 유실, 예약엔 못 쓴다
CDC (이번 설계)	DB 변경 로그로 캐시 갱신	앱 코드와 분리된다	인프라가 복잡하다

인기 키가 만료되는 순간



한 요청만 DB에 보낸다

나머지는 잠깐 기다렸다가 캐시를 다시 본다

만료 시각을 흩뜨린다

TTL에 랜덤 값을 더해 한꺼번에 만료되지 않게

미리 갱신한다

만료 전에 백그라운드에서 새로 채운다

샤드를 16개에서 20개로 늘리면

hotel_id % 16 → hotel_id % 20
-- 대부분의 호텔이 다른 샤드로 옮겨가야 한다



안정 해시

노드를 추가해도
일부 키만 옮겨간다



논리 샤드를 넉넉히

논리 샤드 1,024개를 만들어
물리 DB에 나눠 담는다
늘릴 땐 몇 개만 옮긴다

09 데이터 일관성

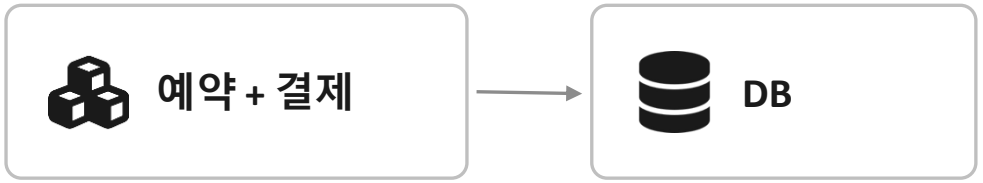


DB가 나뉘면 트랜잭션을 어떻게 묶을까

- 2PC, Saga, Outbox
- 코레오그래피와 오케스트레이션
- 결제의 멱등성

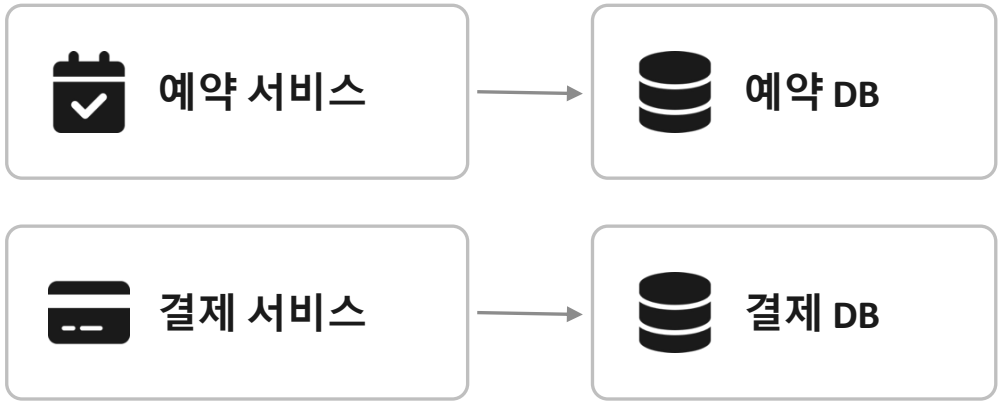
DB가 나뉘면 트랜잭션도 나뉜다

모놀리식



트랜잭션 하나로 끝난다

MSA



예약은 됐는데 결제 기록은 실패하는 상황이 생길 수 있다

모놀리식은 트랜잭션 하나로 끝난다

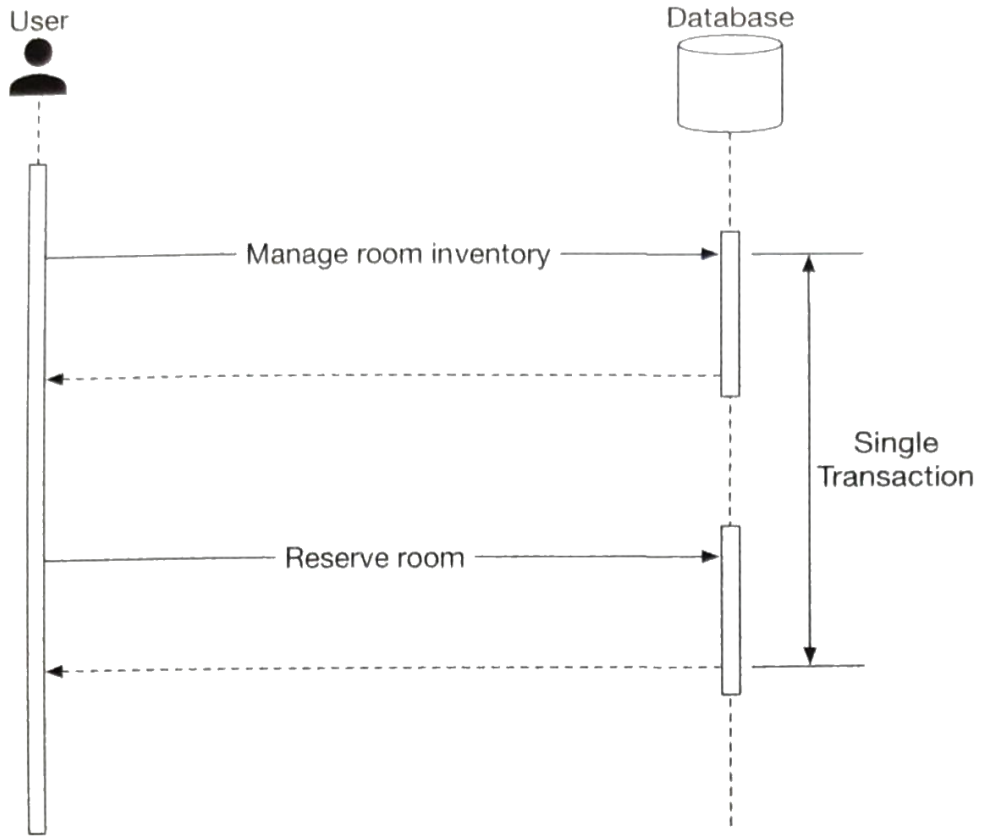
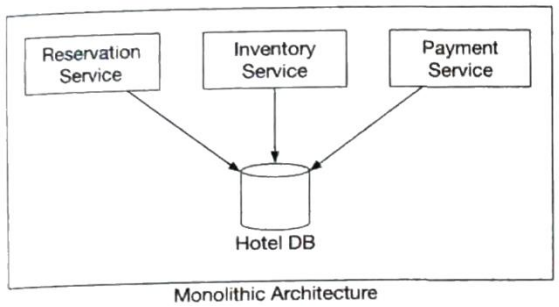
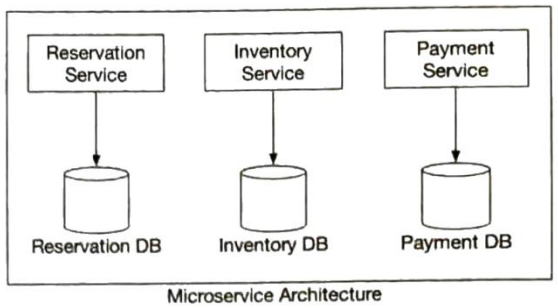


Figure 7.18: Monolithic architecture





2PC, Saga, Outbox

패턴	방식	특징
2PC	코디네이터가 준비 → 커밋 두 단계로 묶는다	일관성은 강하지만 대기가 길고 코디네이터 장애에 약하다
Saga	로컬 트랜잭션을 차례로, 실패하면 보상 트랜잭션	결국 일관된 상태가 된다. 보상 로직을 짜야 한다
Outbox	데이터와 이벤트를 같은 트랜잭션으로 저장 후 발행	저장과 발행이 어긋나지 않는다

✔ **실무에서는 Saga와 Outbox를 같이 쓴다**

저장은 됐는데 이벤트가 사라지면

[예약 서비스] 로컬 트랜잭션 {
INSERT reservation (PENDING)
INSERT outbox (ReservationCreated)
} -- 둘 다 저장되거나 둘 다 안 되거나

릴레이 또는 CDC가 outbox를 읽어 Kafka로 발행

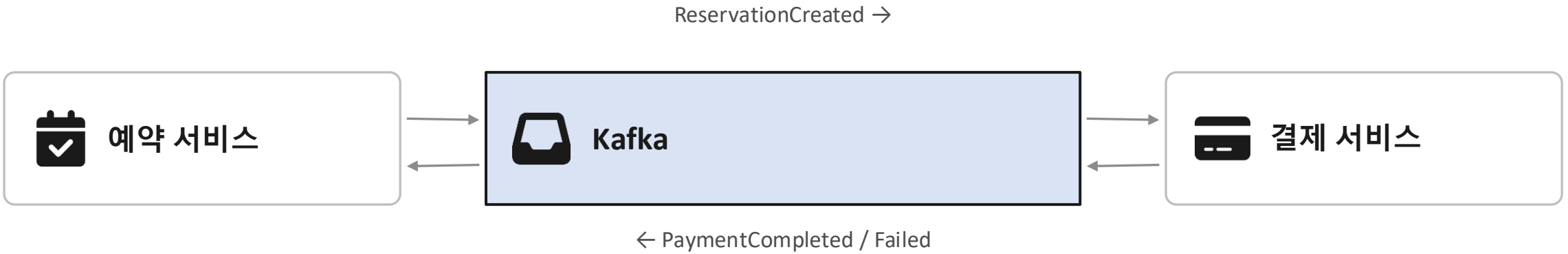
[결제 서비스] 결제 실패 → PaymentFailed
[예약 서비스] 보상: 상태를 REJECTED로, 재고 복구

Saga
흐름을 나누고, 실패하면 되돌린다

Outbox
저장과 발행을 한 번에 묶는다

둘은 서로를 보완한다

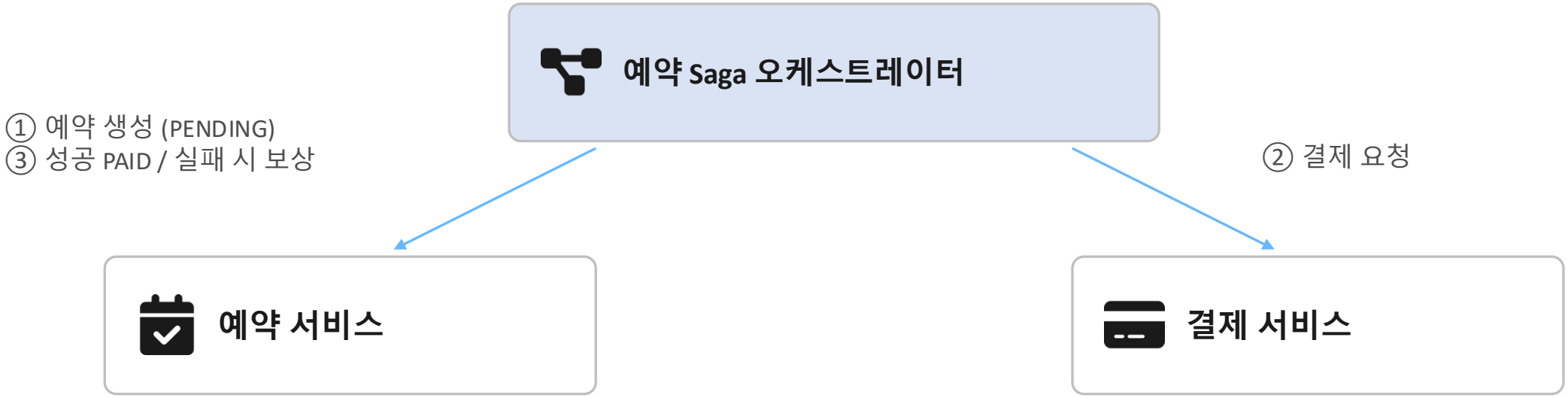
각 서비스가 이벤트를 듣고 움직이는 방식



코레오그래피

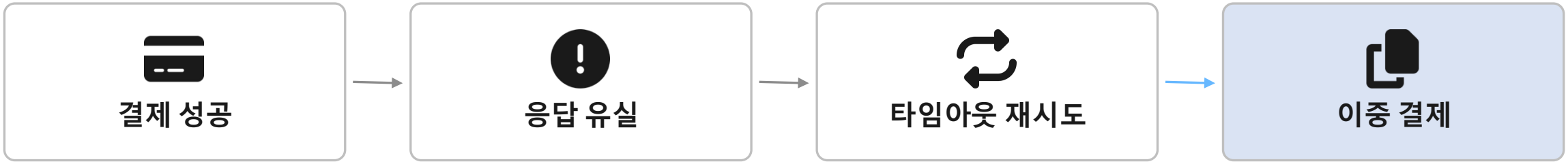
지휘하는 쪽이 없어서 서로 덜 엮이지만, 서비스가 많아지면 흐름을 따라가기 어렵다

오케스트레이터가 순서를 정하는 방식



단계가 적고 단순하면 코레오그래피, 많고 복잡하면 오케스트레이션

결제는 두 번 되면 안 된다



결제 요청에도 멱등 키

같은 키로 오면 이전 결과를 돌려준다
대부분의 PG API가 지원한다

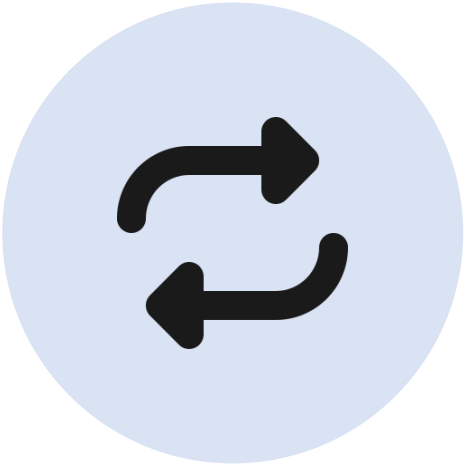
이벤트를 받는 쪽도

Kafka는 보통 최소 한 번 전달
처리한 이벤트 ID를 기록해 중복은 무시한다

기본 가정

모든 요청은
두 번 올 수 있다

클릭도, 재시도도, 이벤트도.
받는 쪽이 멱등해야 한다.



정리하면

읽기 100 : 쓰기 1	관계형 DB와 캐시
재고는 객실 유형 × 날짜마다 한 줄	초과 예약은 1.1배까지
같은 사람은 멱등 키, 다른 사람은 낙관적 락	마지막 판단은 DB
hotel_id 샤딩, CDC로 캐시 갱신	읽기는 캐시, 쓰기는 DB
Saga와 Outbox	틀리면 안 되는 트랜잭션은 한 서비스 안에

처음 질문에 대한 답

멱등 키가 중복을 막고,
낙관적 락이 한 명만 통과시키고,
마지막 판단은 DB가 한다



모든 선택은 ‘읽기 100 : 쓰기 1, 초당 3건’에서 시작했다

같이 생각해 볼 질문



본인의 케이스에 적용해본다면?

이중 처리가 생길 수 있는 곳과, 지금 그걸 막는 방법



재고 원본을 Redis에 두면?

빨라지는 대신 무엇을 포기하게 될까



서비스는 얼마나 잘게 나눠야 할까

예약과 결제를 한 서비스에 두면 쉬운데 왜 나눌까



초과 예약 10%는 누가 정할까

시스템 설계와 비즈니스 정책의 경계

감사합니다